

C++보고서

_rodit예비단원_김선우

1. c++을 왜 사용하나? / 어디에 사용하나?

-C++은 게임분야(ex.unity)와 로봇분야 등 여러 분야에서 주로 사용중

-class를 배운 후, C++은 여러 사람과 협동할 때, 역할을 독립적으로 나눌 수 있는게 장점인 것 같다.

-c언어와 c++의 차이는 c는 절차 지향이고, c++은 객체 지향인 점이다. 이런 차이로 c++은 코드의 규모가 클수록 c보다 유지, 보완하기 편하다.

.2. 하다보니 알게된 것

- ,를 cout에 사용할 때는 '를 써주고, =,+같은 기호는 "를 써준다.

- hw1코드 중에서 delete[] p; //동적할당 메모리 해제 이 부분에 []가 없으면 일부만 동적할당이 해제될 수 있어서 전체 메모리를 해제하기 위해서는 [] 대괄호를 써줘야한다.

- c할 때도 마찬가지로이지만, 좌표 정보를 코드에 쓸 때는 구조체를 사용하는 것이 보기도 좋고, 코드를 쓸 때에도 편한 것 같다.

- .(점)은 해당 구조체나 클래스 내부 변수에 접근할 때 사용하는 연산자

3. 개인적으로 어려운 개념 – 객체, 참조자

- class와 객체

흔한 예로 붕어빵이 어떤 모양인지, 어떤 재료가 들어가는지와 같은 재료, 설계가 class이다. 객체는 붕어빵 틀로 이미 찍어낸 결과물이다. 이는 코드를 보면 이하기 쉽다. Class는 int age, string name과 같이 표현하지만, 객체는 dog mydog

로 객체를 생성하고 mydog.name = "보리"와 같이 표현한다.

-참조자와 복사

복사는 원래 메모리에 있던 데이터를 똑같이 복사해서 새 메모리에 저장한다. 때문에 복사본을 수정해도 원본에는 아무런 영향을 미치지 않는다. 하지만 참조자의 경우, 이미 존재하는 메모리 공간의 주소?를 똑같이 사용한다. 즉, 새 이름/별

명을 부여하면 새 메모리를 만드는 것이 아니라 기존 주소로 찾아가 수정하는 것이다. 따라서 참조자를 사용할 경우, 메모리를 사용하지 않고 원본 코드를 수정할 수 있다.

-생성자와 소멸자

생성자는 객체가 태어날 때 자동으로 실행되는 함수이고, 소멸자는 사라질 때 실행되는 마무리 함수이다. 처음에는 왜 필요하지?라는 생각이 들었지만 동적할당을 쓸 때, 소멸자 안에 `delete[]` 를 해두면 직접 메모리 해제를 하지 않아도 프로그램이 알아서 메모리를 반납하는 안전장치를 구현할 수 있다.

4. 배운 내용을 앞으로 어떻게 활용할 수 있을까?.

- 강의하실 때 메모리 할당, 동적 할당의 중요성을 설명해주셨다. 아직까지는 큰 데이터를 다루지 않아서 문제 없었지만 로봇에서 만드는 로봇 같은 경우에는 로봇에 제한되어 있는 메모리 안에 데이터를 많이 넣게 될 것 같아 동적 할당이 기본적이고 필수적일 것 같다. (ex. 로봇이 공을 집으러 갈 때, 장애물을 마주치면 이동 경로가 상황마다 바뀌지만 이때, `delete[]` 로 해제하지 않으면 많은 메모리를 소비할 수 있다.)
- hw2과제에서 점 사이의 거리를 구하는 과제가 있었다. 로봣 홍보영상에 보면 짧게 나오지만 slam기술을 개발하는 작업도 한다고 한다. 이번 hw2과제를 시작으로 카메라와 라이다로 입력된 정보로 거리를 측정하고 로봇이 실제로 움직이는 작업에도 활용될 수 있을 것 같다.
- class를 통해 private에는 로봇 팔의 모터 각도, 모터 속도 제한 등 하드웨어에 데미지를 줄 수 있는 등 바뀌면 안되는 정보들을 private에 넣어 보관한다.
- 동적할당과 똑같이 로봇의 메모리 사용을 최대한 줄이는 방법으로 코드를 수정할 때, 복사하지 않고 참조자를 활용하여 메모리 사용을 줄이는데 활용해야겠다.

5. 앞으로 어떻게 코딩할 것인가?

- 비록 작은 데이터는 상관없겠지만 최대한 메모리 소비를 줄일 수 있도록 동적할당하는 습관을 만들고, 참조자를 사용하는 습관을 만들어 미래에 메모리를 관리해야할 때 잘 관리할 수 있도록 해야겠다.
- C++에서는 main함수에 많이, 복잡하게 코드를 작성하면 나중에 코드를 수정할 때, 어디서 코드가 틀렸는지 알기 어렵다. 하지만 과제처럼 부분부분을 class의 형태로 기능을

나누고 main함수에는 기본적인 입출력만을 다룬다면 나중에 수정하기가 편해지므로 지금부터 클래스를 나누는 습관을 만들고자한다.

(ex. 하드웨어 제어 담당 class, 센서 데이터 수집 class, 경로 계산 class, 전체적인 class)